

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 2

Roll no: 38 PRN No: 12413458 Name of Student: Vedanti Raut

Subject Name : Deep Learning

Problem Statement:

Design and implement a Multilayer Perceptron (MLP) for classification of the Wine dataset, and evaluate its performance using accuracy and a confusion matrix.

Algorithm :

1. Start.
2. Import the required libraries: TensorFlow, NumPy, Matplotlib, and Scikit-learn modules.
3. Load the Wine dataset using load_wine().
4. Separate the dataset into features (X) and target labels (y).
5. Split the dataset into training and testing sets using train_test_split().
6. Normalize the feature values using StandardScaler.
7. Build a Multilayer Perceptron (MLP) model with:
 - An input layer for 13 features.
 - Two hidden Dense layers with ReLU activation.
 - An output Dense layer with Softmax activation for 3 classes.
8. Compile the model using:
 - Adam optimizer
 - Sparse Categorical Crossentropy loss function
 - Accuracy as the evaluation metric
9. Train the MLP model using the training dataset for a specified number of epochs.
10. Evaluate the trained model using the testing dataset.
11. Predict the class labels for the test data.
12. Calculate and display the accuracy score.

13. Generate and display the classification report (Precision, Recall, F1-score, and Support).
14. Generate and display the confusion matrix.
15. Plot the training and validation accuracy graph.
16. Stop.

Code :

```
# 1. Importing Libraries
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay, classification_report

# 2. Load the Wine Dataset
wine = load_wine()

X = wine.data
y = wine.target

print("Feature Shape :", X.shape)
print("Label Shape :", y.shape)
print("Target Classes :", wine.target_names)
```

```
# 3. Split the Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training Samples :", X_train.shape)
print("Testing Samples :", X_test.shape)

Show hidden output

# 4. Normalize the Data
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 5. Build the MLP Model
model = tf.keras.models.Sequential([
    tf.keras.layers.Dense(64, activation='relu', input_shape=(13,)),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(3, activation='softmax')
])
```

```
# 6. Compile the Model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
# 7. Train the Model
history = model.fit(
    X_train,
    y_train,
    epochs=50,
    validation_split=0.2,
    verbose=1
)
```

Show hidden output

```
# 8. Evaluate the Model
loss, accuracy = model.evaluate(X_test, y_test)

print("Test Loss :", loss)
print("Test Accuracy :", accuracy)
```

```
# 9. Predict Classes
predictions = model.predict(X_test)

predicted_classes = np.argmax(predictions, axis=1)
```

Show hidden output

```
# 10. Calculate Accuracy
acc = accuracy_score(y_test, predicted_classes)

print("Accuracy Score:", acc)
```

Show hidden output

```
# 11. Classification Report
print("\nClassification Report:\n")

print(classification_report(
    y_test,
    predicted_classes,
    target_names=wine.target_names
))
```

```
# 12. Confusion Matrix
cm = confusion_matrix(y_test, predicted_classes)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=wine.target_names
)

disp.plot(cmap='Blues')
plt.title("Confusion Matrix")
plt.show()
```

Show hidden output

```
# 13. Plot Training Accuracy
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training vs Validation Accuracy")
plt.legend()
plt.grid(True)
plt.show()
```

Output :

2. Load the Wine Dataset

```
Feature Shape : (178, 13)
Label Shape : (178,)
Target Classes : ['class_0' 'class_1' 'class_2']
```

3. Split the Dataset

```
Training Samples : (142, 13)
Testing Samples : (36, 13)
```

8. Evaluate the Model

```
2/2 ————— 0s 28ms/step - accuracy: 1.0000 - loss: 0.0200
Test Loss : 0.020032711327075958
Test Accuracy : 1.0
```

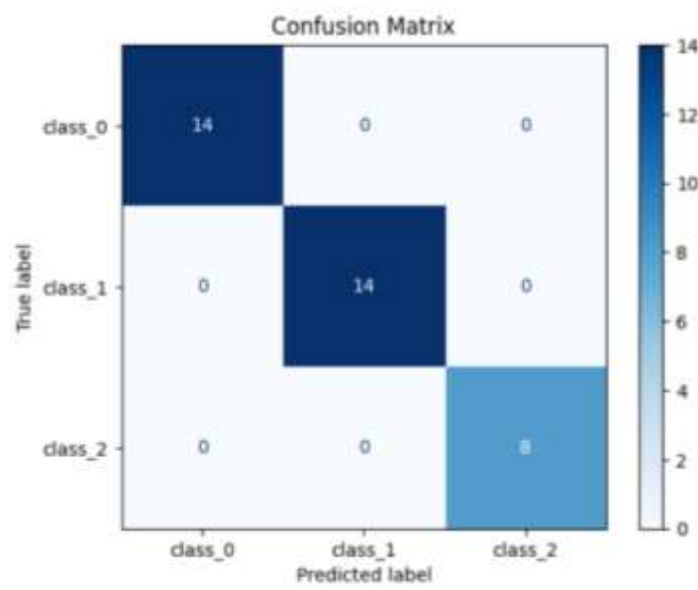
10. Calculate Accuracy

```
Accuracy Score: 1.0
```

11. Classification Report

Classification Report:				
	precision	recall	f1-score	support
class_0	1.00	1.00	1.00	14
class_1	1.00	1.00	1.00	14
class_2	1.00	1.00	1.00	8
accuracy			1.00	36
macro avg	1.00	1.00	1.00	36
weighted avg	1.00	1.00	1.00	36

12. Confusion Matrix



13. Plot Training Accuracy

